Example topics/questions for Theory test 1 – 2026 Autumn

Both theory tests and their retakes will be in multiple-choice format. For each question, there will typically be 3 to 6 possible statements, of which AT LEAST ONE will be true. To receive a full score for a question, you must correctly select all true statements and only those. Half score is given for a specific question if exactly one mistake was made. Note, that selecting an incorrect answer and not selecting a correct answer at the same time counts as two mistakes (e.g., selecting A,B instead of A,C counts as two mistakes, and is therefore not worth any points).

Below a list of example questions/topics are given. The examples are not in the multiple-choice format here, however, expect multiple-choice format questions during the test with similar topics. This list is not exhaustive: it provides only examples.

<u>Example questions related to Lectures 1 - 6</u>

- What are the inputs to the hypothesis function?
- What are the outputs (the value) of the hypothesis function?
- What can be the format of a label? (scalar, vector, matrix?)
- What are the constituents of a data point in supervised learning?
- What is the role of the loss function?
- Can the value of the loss function be a scalar / vector / matrix?
- In reinforcement learning, what form of feedback do we get from the environment? How are these different from feedback in supervised learning?
- Given different machine learning tasks (regression, classification, clustering, dimensionality reduction, denoising)
    - Which of these tasks are in the domain of supervised learning and which of them are in unsupervised learning?
    - What is the goal in these tasks?
- Is linear regression used to solve regression or classification tasks? Is it a supervised learning method or an unsupervised learning method?
- Is logistic regression used to solve regression or classification tasks? Is it a supervised learning method or an unsupervised learning method?
- Given an example task: We would like to predict the cholesterol level of patients based on their personal attributes: weight, age, height, sex. We would like to train a model on our training set which contains 200 patients, each with the 4 attributes mentioned above plus the cholesterol level values provided.
    - Is linear regression or logistic regression a more suitable choice for this task?
    - If we use linear regression, how many parameters will we have?
    - What loss function should we use?
    - In case, the hypothesis function of a linear regression model solving this task is $\widehat{y} = X\theta$ where X is a matrix of all training examples …

        - what is the shape of the X matrix?
        - what is the shape of the θ vector?
        - what will be the shape of the $\widehat{y}$ vector?
        - what is the range of values in the $\widehat{y}$ vector?
        - are the age values of patients inside θ or X?
        - do we want to minimize X during learning?
        - do we want to minimize y during learning?
        - do we want to minimize θ during learning?
    - In case, the hypothesis function of a linear regression model solving this task is $\widehat{y} = \theta_4 x_4 + \theta_3 x_3 + \theta_2 x_2 + \theta_1 x_1 + \theta_0$ where $\widehat{y}$ is a scalar:
        - Is θ_0 a vector or a scalar?
        - What can be the range of the value of $\widehat{y}$?
        - Is the age value of a patient assigned to one of the x_i values or one of the θ_i values?
    - (Expect questions on the geometric interpretation of the hypothesis function of linear regression:
        - If there is 1 input variable, and we visualize the data points on a 2D plot with the x, y axes, what geometric object will be the hypothesis function?
        - If there are 2 input variables, and we visualize the data points on a 3D plot with the x_1, x_2, y axes, what geometric object will be the hypothesis function?)
    - In case we want to solve this task using linear regression with least squares (MSE loss) and we would like to use the (regularized) normal equation to find the optimal parameters:

$$\theta = (X^T X + \lambda I)^{-1} X^T y$$

        - What is the shape of the X matrix?
        - What is the shape of the y vector?
        - What is the shape of the θ vector?
        - Are the cholesterol levels placed in X, y or θ?
        - What is the I matrix?
- In case of linear regression with the MSE loss function, are we guaranteed to (approximately) find the globally optimal parameters by gradient descent, given a small enough step size?
- In the case of a Multi-Layer Perceptron with the MSE loss function, are we guaranteed to (approximately) find the globally optimal parameters by gradient descent, given a small enough step size?
- What is the purpose of feature scaling?
- What may be more likely to happen if we do not use feature scaling on the input variables?

- Which feature scaling technique of min-max scaling and standardization is more robust to a small number of outlier data points?
- Given the formula for one of the feature scaling techniques we discussed, what are the constituents of the formula? (Identifying various symbols and their roles in the formulas.)
- Given an example task: We would like to distinguish patients with a certain illness from healthy persons based on two of their laboratory blood test result values. We would like to train a model on our training set which contains 100 patients, each with the 2 attributes mentioned above plus their label storing whether they have the illness or not.
    - Is linear regression or logistic regression a more suitable choice for this task?
    - If we use a logistic regression, how many parameters will we have?
    - What loss function should we use?
    - In case, the hypothesis function of a logistic regression model solving this task is $\hat{y} = g(X\theta)$ where g is the sigmoid function and X is a matrix of all training examples:
        - What is the shape of the X matrix?
        - What is the shape of the θ vector?
        - What will be the shape of the $\hat{y}$ vector?
        - What is the range of values in the $\hat{y}$ vector?
        - Are the laboratory blood test result values of patients inside θ or X?
        - Do we want to minimize X during learning?
        - Do we want to minimize y during learning?
        - Do we want to minimize θ during learning?
        - Do we want to minimize g during learning?
        - What do we want to minimize during learning?
    - In case, the hypothesis function of a logistic regression model solving this task is $\hat{y} = g(\theta_2 x_2 + \theta_1 x_1 + \theta_0)$ where g is the sigmoid function and $\hat{y}$ is a scalar:
        - Is θ_0 a vector or a scalar?
        - What can be the range of the value of $\hat{y}$?
        - What can be the range of the value of $g(\theta_2 x_2 + \theta_1 x_1 + \theta_0)$?
        - Are the laboratory blood test result values of patients assigned to one of the x_i values or one of the θ_i values?
- Consider the following loss function:

$$J(\Theta) = \frac{1}{2m} \sum_{j=1}^{m} (\hat{y}^{(j)} - y^{(j)})^2$$

    - Is this loss function more suitable to be used for a regression or a classification task?
    - The variable j in the formula is indexing what?
    - What is m in the formula?
    - Is J(Θ) (the value of the function) a scalar, a vector or a matrix?
    - (Similar questions with other loss functions…)

- Consider the following hypothesis function of a Multi-Layer Perceptron:

  $$h(x) = g_2(W_2\ g_1(W_1 x + b_1) + b_2)$$

  - We would like to use this MLP to solve a regression task with 20 input variables and we would like to estimate a single continuous value.
    - In case we have 10 neurons in the first layer, how many parameters are there in the whole network?
    - In case we have 10 neurons in the first layer, how many parameters are there in the first layer?
    - Should we omit the g1 activation function?
    - Should we omit the g2 activation function?
    - Should g2 be a sigmoid activation function?
  - (Similar questions for an example binary classification task)
  - (Similar questions for an example multi-class classification task)
- (Additional questions on the properties of activation functions: identifying them visually from their graphs, questions about the range of values on the output, etc.)

- If we decrease the number of training examples when training a given neural network model, do we expect overfitting to become more severe?
- If we decrease the number of test examples when training a given neural network model, do we expect overfitting to become more severe?
- If we increase the number of training examples when training a given neural network model, do we expect overfitting to become more severe?
- If we decrease the number of parameters in our model when training a neural network, do we expect overfitting to become more severe?
- If we increase the number of parameters in our model when training a neural network, do we expect overfitting to become more severe?
- For what reason do we use the early stopping method?
- When should we stop the training according to the early stopping method?
- Should we stop the training when the training loss begins to increase according to the early stopping method?
- If the validation loss keeps increasing while the training loss keeps decreasing during the training of our model, can these be interpreted as signs of an overfitting, getting more severe?
- Are the parameters of a neural network usually trained with a gradient descent based algorithm (like SGD)?
- Are the hyper-parameters of a neural network usually trained with a gradient descent based algorithm (like SGD)?

- Is the SGD algorithm such a variant of the gradient descent algorithm where we compute the gradient on a single training example in each step, instead of a batch of examples?
- What effects can be expected if we increase the batch size in mini-batch SGD?
- What effects can be expected if we decrease the batch size in mini-batch SGD?

- Given the computational graph of the expression J, containing a "square" node, e.g., c = b^2. Let grad_c be the partial derivative of the expression J with respect to c (so ∂J/∂c). Let grad_b be the partial derivative of the expression J with respect to b (so ∂J/∂b). Using the backpropagation algorithm for gradient backpropagation, what do we propagate back in the direction of b, i.e., what is grad_b?
  Example answers (either true or false):
  - grad_b = grad_c * 2 * c
  - grad_b = grad_c * 2 * b
  - grad_b = grad_c * c^2
  - grad_b = grad_c
- Given the computational graph of the expression J, containing an "addition" node, e.g., c = a + b. Let grad_c be the partial derivative of the expression J with respect to c (so ∂J/∂c). Let grad_a and grad_b be the partial derivatives of the expression J with respect to a and b, respectively (so ∂J/∂a and ∂J/∂b, respectively). Using the backpropagation algorithm for gradient backpropagation, what do we propagate back in the direction of a, and what in the direction of b, i.e., what will grad_a and grad_b be?
  - grad_a = grad_c + b, grad_b = grad_c + a
  - grad_a = grad_b = grad_c
  - grad_a = grad_c + a, grad_b = grad_c + b

Example questions related to NumPy

(Questions will be based on the content of the videos "Video to watch before practice" for weeks 3-6, only related to NumPy. The concepts of ndarray.strides, stride tricks will not be included. The list of questions below is not exhaustive, it only provides a few examples.)

- If `my_array` is a numpy.ndarray instance:
  - Can `my_array.shape` be equal to (5, 6)?
  - Can `my_array.shape` be equal to (1,)?
  - Can `my_array.shape` be equal to (0, 1, 0)?
  - Can `my_array.shape` be equal to 12?
  - Can `my_array.dtype` be equal to (5, 6)?
  - Can `my_array.size` be equal to (5, 6)?
  - Can `my_array.size` be equal to 12?
  - Can `my_array.ndim` be equal to (5, 6)?

- If `my_array` is a numpy.ndarray instance with a shape of (32, 5):
  - What will be the shape of `my_array[:, :]`?
  - What will be the shape of `my_array[:, 1]`?
  - What will be the shape of `my_array[1, :]`?
  - What will be the shape of `my_array[1:, :]`?
  - What will be the shape of `my_array[0, :-1]`?

    - What will be the shape of `my_array[[0], :]`?
    - What will be the shape of `my_array[[-1, -1], -1]`?

- if `my_array1` is a numpy.ndarray instance with a shape of (32, 5) and `my_array2` is a numpy.ndarray instance with a shape of (32,), both of numeric data types:
    - Will `my_array1 + my_array2` raise an error "`ValueError: operands could not be broadcast together …`"?
    - Will `my_array1 + my_array2[:, None]` raise an error "`ValueError: operands could not be broadcast together …`"?
    - Will `my_array1 + my_array2[None, :]` raise an error "`ValueError: operands could not be broadcast together …`"?
    - Will `my_array1 + 1` raise an error "`ValueError: operands could not be broadcast together …`"?

- If `my_array` is a numpy.ndarray instance with a shape of (32, 5):
    - What will be the shape of `my_array < 0`?
    - What will be the shape of `my_array.T`?
    - What will be the shape of `my_array.T.T`?
    - What will be the shape of `my_array + my_array`?
    - What will be the shape of `my_array + my_array[:,None,:]`?
    - What will be the shape of `np.mean(my_array, axis=0)`?
    - What will be the shape of `np.mean(my_array, axis=1)`?
    - Will the type of `np.sum(my_array, axis=0)` be numpy.ndarray?
    - Will the type of `np.sum(my_array)` be numpy.ndarray?

- Let `my_array` be defined in the following way: `my_array = np.arange(4).reshape(2, -1)`
    - What will be the shape of `my_array`?
    - What will be the shape of `my_array * 2 + 1`?
    - What will be the shape of `my_array[my_array > 0]`?
    - What will be the shape of `my_array[(my_array > 0) & (my_array % 2 == 0)]`?
    - What will be the shape of `my_array * my_array.reshape(2, 2, -1)`?

- Let `my_array` be defined in the following way: `my_array = np.arange(4)`
    - Let `my_array2` be defined in the following way: `my_array2 = my_array.reshape(2, 2)`. Will the elements of `my_array` be affected if we execute `my_array2[:, :] = 99`?

    - Let `my_array2` be defined in the following way: `my_array2 = my_array.T`. Will the elements of `my_array` be affected if we execute `my_array2[:] = 99`?
    - Let `my_array2` be defined in the following way: `my_array2 = np.append(my_array, 42)`. Will the elements of `my_array` be affected if we execute `my_array2[:] = 99`?
    - Let `my_array2` be defined in the following way: `my_array2 = my_array[1:2]`. Will the elements of `my_array` be affected if we execute `my_array2[:] = 99`?
    - Let `my_array2` be defined in the following way: `my_array2 = my_array[[1, 2]]`. Will the elements of `my_array` be affected if we execute `my_array2[:] = 99`?
    - Let `my_array2` be defined in the following way: `my_array2 = my_array[my_array < 2]`. Will the elements of `my_array` be affected if we execute `my_array2[:] = 99`?
    - Let `my_array2` be defined in the following way: `my_array2 = my_array + my_array`. Will the elements of `my_array` be affected if we execute `my_array2[:] = 99`?

- Let `my_array` be defined in the following way: `my_array = np.arange(4).reshape(2, -1)`
    - What will be the result of `np.count_nonzero(my_array == 0)`?
    - What will be the result of `np.any(my_array == 0)`?
    - What will be the result of `np.all(my_array == 0, axis=1)`?

- Let `my_array` be defined in the following way: `my_array = np.array([3, 1, 2])`
    - What will be the result of `np.unique(my_array)`?
    - What will be the result of `np.unique(my_array == 0)`?
    - What will be the result of `np.bincount(my_array)`?
    - What will be the result of `np.sort(my_array)[::-1]`?
    - What will be the result of `np.argsort(my_array)`?
    - What will be the result of `my_array[my_array != 0]`?